FastAPI Notes

Personal study notes - FastAPI-Udemy

Last updated: 28 October 2026

Contents

1. Getting Started
2. Your First App (main.py)
3. Documenting Endpoints (main1.py)
4. The Request Object
5. Sync vs Async
6. Built-in Tooling
7. Learning Log

1. Getting Started

What FastAPI is

- A modern Python web framework for building APIs, built on **Starlette** (web layer) and **Pydantic** (data validation).
- Automatically generates interactive API docs from your code.
- Served by an ASGI server such as **uvicorn**.

Project setup

- Create a virtual environment so dependencies stay isolated per project.
- Install with `pip install "fastapi[standard]"` and freeze with `pip freeze > requirements.txt`.
- Never commit `venv/` - add it to `.gitignore`.

```
python3 -m venv venv
source venv/bin/activate
pip install "fastapi[standard]"
```

Running the app

- `uvicorn main:app --reload` means: file `main.py`, object `app`, auto-restart on code change.
- `fastapi dev main.py` is the newer CLI alternative.
- Run a different file by changing the module name, e.g. `uvicorn main1:app --reload`.

2. Your First App (main.py)

Creating the app

- `app = FastAPI(...)` is the central object; every route attaches to it.
- Metadata passed to the constructor shows up in the docs UI.

Argument	Purpose
title	Name shown at the top of the docs
description	Longer text under the title
version	API version string
docs_url	Swagger UI path (default /docs)
redoc_url	ReDoc path (default /redoc)
openapi_url	Raw OpenAPI schema path (default /openapi.json)

```
app = FastAPI(
    title="My First FastAPI App",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
    openapi_url="/openapi.json",
)
```

Your first route

- `@app.get("/")` registers the function below it as the handler for GET requests to `/`.
- Returning a Python `dict` is automatically converted to JSON.

```
@app.get("/")
def basic_func():
    return {"message": "Hello there !"}
```

3. Documenting Endpoints (main1.py)

App-level metadata

- `summary` is a one-line tagline; `description` is the longer text.
- Wrap long strings in parentheses to split them across lines (adjacent strings are concatenated - remember to add spaces between pieces).

Tags

- `tags=["name"]` groups endpoints together in the docs UI.
- An endpoint can have several tags and will appear under each.

Docstrings

- The function docstring becomes the endpoint description in the docs.

```
@app.get("/", tags=["there"])
def home():
    """Doc string to get to know about the function"""
    return {"home": "hello you are at home"}
```

Everything the decorator can document

- The decorator can describe the endpoint itself, not just its path.
- `summary` - short title of the operation.
- `description` - long explanation (overrides the docstring).
- `deprecated=True` - shows the endpoint as struck out in the docs.
- `tags` - grouping.

```
@app.get(
    "/MoreInfo",
    summary="This is more about my request",
    description="Hello there",
    deprecated=False,
    tags=["hello", "there", "request"],
)
async def Func(request: Request): ...
```

4. The Request Object

Inspecting the incoming request

- Declare a parameter typed as `Request` (from `fastapi`) and FastAPI injects it.
- Useful attributes:

Attribute	Gives you
request.headers	All HTTP headers (dict-like)
request.url	Full URL requested
request.method	GET, POST, ...
request.path_params	Values captured from the path
request.query_params	Values after `?` in the URL

```
@app.get("/request", tags=["hello"])
async def req_func(req: Request):
    return {
        "headers": dict(req.headers),
        "url": str(req.url),
        "method": req.method,
    }
```

Notes

- Convert non-JSON types (URL, QueryParams) with `str()` / `dict()` before returning.
- Try `/request?name=sam&age=10` and watch `query_params` change.

5. Sync vs Async

- `def` handlers run in a thread pool; `async def` handlers run on the event loop.
- Use `async def` when you `await` things (DB drivers, HTTP clients). Use plain `def` for blocking code.
- Never call blocking code (like `time.sleep`) inside `async def` - it freezes the whole server.

6. Built-in Tooling

Swagger UI and ReDoc

- `/docs` - interactive Swagger UI, lets you try requests in the browser.
- `/redoc` - read-only, clean reference docs.
- `/openapi.json` - machine-readable schema; used by Postman and code generators.

Postman

- This repo has a `postman/` folder linked to a Postman workspace (collections, environments, specs).
- Import `/openapi.json` into Postman to generate a collection automatically.

7. Learning Log

Add new things you learn below. Each time you ask for an update, they are folded into the chapters above and the PDF is rebuilt.

- (nothing pending)